COER UNIVERSITY, ROORKEE

Assignment-4: Software Testing and Quality Assurance

MCA 2nd Year | Subject Code: MCA211D

Name: **Srishti Malik** Roll No.: **253028081**

Q1. What is a Defect? Explain the importance of defect tracking in software testing.

A **defect** (also called a **bug** or **fault**) is a flaw in a software component or system that causes it to behave differently from what is expected or specified. It exists when the actual result of the software does not match the expected result defined in the requirements, design or user expectations.

The related terms are often confused:

- **Error (mistake):** a human action that produces an incorrect result, e.g., a programmer misunderstands a requirement.
- **Defect (bug/fault):** the flaw in the code or document that results from the error.
- **Failure:** the visible incorrect behaviour of the software when the defective code is executed.

Error (human mistake) → Defect (fault in code/document) → Failure (wrong behaviour at run time)

Common causes of defects: unclear or changing requirements, design mistakes, coding errors, poor communication, lack of testing or reviews, time pressure and complex technology.

Importance of defect tracking

Defect tracking is the process of recording every defect and following it until it is resolved and closed. Its importance is:

- **Nothing is lost:** Every defect is recorded, so none is forgotten or ignored.
- **Clear ownership:** Each defect is assigned to a person with a status, so responsibility is visible.
- **Prioritisation:** Critical defects can be identified and fixed before minor ones.
- **Better communication:** Testers, developers and managers share one record instead of informal messages.
- **Progress monitoring:** Managers can see how many defects are open, fixed or closed.
- **Quality measurement:** Data on defects gives metrics such as defect density and defect removal efficiency.
- **Release decisions:** Open defect counts and severities help decide whether the product is ready to be released.
- **Root cause analysis and prevention:** Defect history shows recurring problems and helps improve development and testing processes.
- **Reuse of knowledge:** Past defects and their fixes are a reference for future projects and regression testing.
- **Accountability and audit:** Provides evidence of testing and fixing for audits and customers.

Q2. Explain the Defect Life Cycle with a suitable diagram.

The **Defect Life Cycle (Bug Life Cycle)** is the journey of a defect through various states from the moment it is discovered until it is finally closed. It makes sure that each defect is reviewed, fixed and verified in a controlled

and trackable way. The exact states differ from one organisation or tool to another, but the general flow is shown in the diagram below.

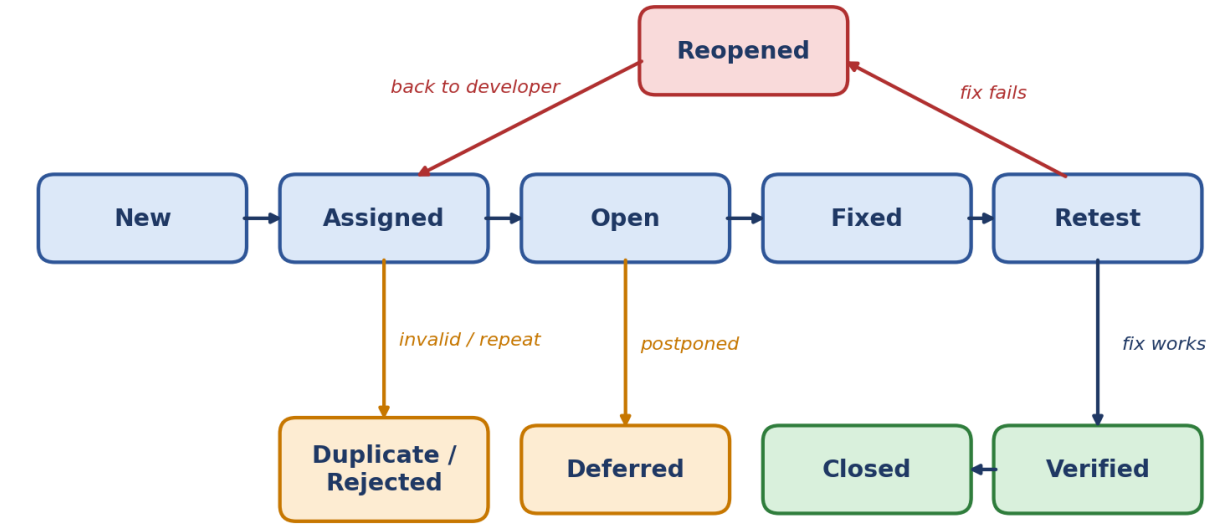


Fig. 1: Defect Life Cycle

Explanation of the flow

1. The tester finds a defect and logs it in the tracking tool with the status **New**.
2. The test/project lead reviews it. If it is valid, it is **Assigned** to a developer; if it is invalid or already reported, it is marked **Rejected** or **Duplicate**; if it is of low priority, it may be **Deferred** to a later release.
3. The developer starts analysing the defect and the status becomes **Open**.
4. After correcting the code, the developer marks it **Fixed** and delivers the fix in a new build.
5. The tester re-executes the failed test case in the **Retest** state.
6. If the defect is gone and no side effects are found, it is marked **Verified** and then **Closed**.
7. If the defect still exists, it is marked **Reopened** and goes back to the developer, so the cycle repeats until it is fixed.

Q3. What are the different states of a defect in the Defect Life Cycle? Explain each state.

State	Explanation	Done by
New	A defect has just been found and logged for the first time. It has not yet been reviewed or approved.	Tester
Assigned	The defect has been reviewed, accepted as valid and assigned to a developer or team for correction.	Test / project lead
Open	The developer has started analysing the defect and working on the solution.	Developer
Fixed	The developer has corrected the code and the fix is ready for testing.	Developer

State	Explanation	Done by
Pending Retest	The fix has been released in a new build and is waiting to be retested.	Developer / build team
Retest	The tester re-executes the test case to check if the defect has really been removed.	Tester
Verified	Retesting passed: the defect no longer occurs and no new problems were caused by the fix.	Tester
Closed	The defect is confirmed as resolved and no further action is needed. This is the final state.	Tester / lead
Reopened	The defect still exists (or has reappeared) after the fix, so it is sent back to the developer.	Tester
Rejected	The developer or lead decides that it is not a defect (the software works as designed or the report is wrong).	Developer / lead
Duplicate	The same defect has already been reported; this report is linked to the original and closed.	Lead / developer
Deferred (Postponed)	The fix is postponed to a future release because it is of low priority/impact or the time is short.	Project manager / lead
Not Reproducible	The developer cannot reproduce the defect with the given steps; more information is requested from the tester.	Developer
Cannot Fix	The defect cannot be fixed because of technical limitations or third-party constraints.	Developer / lead

Q4. Explain the process of Defect Tracking and Reporting in software testing.

Defect tracking means monitoring every defect from its discovery to its closure, and **defect reporting** means documenting and communicating defect information to the concerned people. The process generally consists of the following steps:

1. **Discover the defect:** While executing test cases, the tester notices that the actual result differs from the expected result.
2. **Confirm and reproduce:** The tester repeats the steps to make sure the defect is genuine and reproducible, and checks whether it is already reported.
3. **Log the defect:** A defect report is created in the tracking tool with a unique ID, summary, steps to reproduce, expected and actual results, environment, severity and attachments.
4. **Classify:** The defect is categorised by type (functional, UI, performance, security), severity and module.
5. **Review and triage:** A defect triage meeting (testers, developers, managers) decides validity, priority and who will fix it.
6. **Assign:** The defect is assigned to a developer with a target date for the fix.
7. **Fix:** The developer analyses the cause, corrects the code, performs unit testing and updates the status to Fixed.
8. **Retest and regression test:** The tester verifies the fix in the new build and checks that nothing else is broken.
9. **Close or reopen:** If the fix works, the defect is closed; otherwise it is reopened with comments.

10. **Report and analyse:** Status reports and metrics (open/closed defects, defect trend, severity distribution, defect age) are generated and shared with the team and management.

Reporting is done through daily/weekly defect status reports, dashboards, defect summary reports and defect-triage meetings, so that all stakeholders know the current quality status of the product.

Q5. What information should be included in a Defect Report? Explain.

A defect report must contain enough information for the developer to understand, reproduce and fix the problem without asking the tester again. The important fields are:

Field	Explanation
Defect ID	Unique number generated by the tool for identification and tracking.
Title / Summary	A short, clear one-line statement of the problem (e.g., "Login fails with valid password containing special character").
Description	A detailed explanation of the defect and where it occurs.
Project / Module	Application and module or feature in which the defect is found.
Build / Version	The software build or release in which the defect was detected.
Steps to Reproduce	Exact numbered steps that lead to the defect, with test data used.
Expected Result	The correct behaviour according to the requirement.
Actual Result	The incorrect behaviour that was observed.
Environment	Operating system, browser/device, database, network and configuration details.
Severity	The seriousness of the defect's impact on the system.
Priority	The urgency with which the defect should be fixed.
Defect Type / Category	Functional, UI, performance, security, compatibility, documentation, etc.
Status	Current state of the defect in the life cycle.
Attachments	Screenshots, screen recordings, log files and error messages as evidence.
Reported by / Date	Name of the tester and date/time of reporting.
Assigned to	The developer or team responsible for the fix.
Comments / History	Discussion, clarifications and changes made over time.
Fix details (by developer)	Root cause, fix description, fixed-in build and date.

Sample defect report

Field	Value
Defect ID	DEF-0125
Summary	Payment fails when the card expiry month is 12
Module / Build	Checkout / v2.3.1
Steps to reproduce	1. Add an item to the cart 2. Go to Checkout 3. Enter a valid card with expiry 12/28 4. Click Pay
Expected result	Payment is processed and a confirmation page is shown
Actual result	Error "Invalid expiry date" is displayed

Field	Value
Environment	Windows 11, Chrome 120, Test server
Severity / Priority	Major / High
Status	New

Q6. Differentiate between Defect Severity and Defect Priority with suitable examples.

Defect severity is the degree of impact the defect has on the functioning of the software (a technical measure). **Defect priority** is the order in which the defect should be fixed, based on business need and urgency (a business measure).

Basis	Severity	Priority
Meaning	How seriously the defect affects the system functionality.	How soon the defect should be fixed.
Based on	Technical impact on the application.	Business value, customer need and deadlines.
Decided by	Usually the tester / QA engineer.	Usually the product owner / project manager (in consultation with the lead).
Focus	Quality of the product.	Scheduling of the fix.
Levels	Critical, Major, Minor, Trivial.	High (urgent), Medium, Low.
Changes over time	Generally remains fixed.	May change with the release schedule and business needs.
Question answered	"How bad is it?"	"How soon should it be fixed?"

Examples of the combinations

Combination	Example
High severity – High priority	The application crashes when a user clicks "Pay Now" on the checkout page.
High severity – Low priority	The application crashes in a rarely used feature, e.g., when a very old file format is imported.
Low severity – High priority	The company name or logo is spelled wrongly on the home page (no functional impact, but harmful to the brand and needs an immediate fix).
Low severity – Low priority	A minor spelling mistake or alignment issue in the help page of an internal module.

Q7. Explain the Defect Management Process in detail.

Defect Management is the systematic process of identifying, recording, classifying, resolving and preventing defects so that the quality of the software improves. The aim is to find defects early, fix them at minimum cost and learn from them to avoid similar defects in future. The process is commonly described in the following stages:

1. **Defect Prevention:** Activities that reduce the chance of defects being introduced, such as reviewing requirements and design, coding standards, training, static analysis and root cause analysis of earlier defects. Preventing a defect is cheaper than fixing it.

2. **Deliverable Baseline:** A deliverable (requirements document, design, code module) is baselined once it is reviewed and approved. After this point, any deviation from the baseline is considered a defect and is managed formally.
3. **Defect Discovery:** Defects are found through reviews, inspections and testing. The tester identifies the defect, reproduces it, and reports it. The development team acknowledges whether it is valid.
4. **Defect Resolution:** The defect is prioritised, scheduled and assigned to a developer. The developer fixes it and notifies the tester, who retests the fix and either closes or reopens the defect.
5. **Process Improvement:** Defect data are analysed to find root causes and weak areas of the development and testing process. Improvements (training, better reviews, new tools) are then introduced.
6. **Management Reporting:** Defect information, status and metrics are reported to management and stakeholders to support decisions on release, risk and resources.

Prevention → Baseline → Discovery → Resolution → Process Improvement → Management Reporting

Q8. What are the main activities involved in Defect Management? Explain each activity.

Activity	Explanation
1. Defect Identification	Finding deviations from expected behaviour through testing, reviews, inspections and user feedback.
2. Defect Logging / Recording	Documenting the defect with all required details in the defect tracking system.
3. Classification and Categorisation	Grouping defects by type, module, phase of origin, severity and cause, which helps analysis.
4. Prioritisation and Triage	Reviewing new defects in triage meetings, deciding validity, severity, priority and the target release.
5. Assignment	Allocating the defect to a responsible developer or team with a due date.
6. Defect Resolution (Fixing)	The developer analyses the root cause, corrects the code, performs unit testing and delivers the fix.
7. Verification (Retesting and Regression)	The tester confirms that the defect is fixed and that the fix has not broken other features.
8. Closure	Closing verified defects, or reopening them if the problem remains; rejecting invalid and duplicate defects.
9. Monitoring and Tracking	Following the status, ageing and workload of defects to ensure timely resolution and escalate delays.
10. Reporting and Metrics Analysis	Preparing defect reports and metrics (density, trend, age, DRE) for team and management.
11. Root Cause Analysis and Prevention	Studying why defects occurred and taking actions so that similar defects do not occur again.

Q9. Explain the role of Defect Management Tools in software testing.

Defect management tools (defect tracking tools or bug trackers) are software applications that help teams record, track, manage and report defects. Examples are **Jira, Bugzilla, Mantis, Redmine, Azure DevOps, Trello and YouTrack**.

Role of defect management tools

- **Central repository:** All defects are stored in one place that is accessible to the whole team, replacing emails and spreadsheets.
- **Standardised reporting:** Templates ensure every defect is reported with complete and consistent information.
- **Workflow automation:** The tool manages the defect life cycle, assignment, status changes and notifications.
- **Tracking and visibility:** Everyone can see the status, owner and history of any defect at any time.
- **Better collaboration:** Testers, developers and managers can comment, attach files and communicate on each defect.
- **Prioritisation and scheduling:** Helps sort defects by severity, priority and release.
- **Reports, dashboards and metrics:** Automatically generates defect trends, ageing, density, and open/closed charts.
- **Traceability:** Links defects to test cases, requirements and builds.
- **Integration:** Works with test management tools, version control and CI/CD pipelines.
- **Audit trail and historical data:** Maintains a complete history for audits, analysis and future projects.
- **Saves time and reduces errors:** Avoids duplicate reports and lost defects.

Q10. What are the important features of a good Defect Tracking Tool? Explain.

Feature	Explanation
Easy defect logging	A simple, customisable defect form with fields for all required information and support for attachments such as screenshots and logs.
Customisable workflow	The life-cycle states, transitions and permissions can be adapted to the organisation's process.
User roles and access control	Different permissions for testers, developers, managers and clients to protect data and control actions.
Search, filter and sort	Powerful queries to find defects by status, severity, module, assignee or date; duplicate detection.
Notifications and alerts	Automatic emails/messages when a defect is assigned, updated or overdue.
Reporting and dashboards	Ready-made and custom reports, charts and metrics for status, trends and ageing.
Traceability and integration	Links to test cases, requirements and code commits; integration with test management, version control and CI/CD tools.
History and audit trail	A record of every change, comment and status transition.
Collaboration	Comments, mentions and discussion threads on each defect.
Scalability and performance	Can handle many projects, users and defects without slowing down.
Security and backup	Secure login, data protection and backup/recovery.
Usability and accessibility	A clear, user-friendly interface with web and mobile access; little training needed.
Export and import	Export of data to Excel/PDF/CSV and import from other tools.
Cost and support	Reasonable licensing cost, good documentation and vendor/community support.

Q11. Explain how defects are identified, recorded, assigned, fixed, retested, and closed during the Defect Management Process.

The following steps follow one defect through the whole process. An example is added to make each step concrete: *a "Forgot Password" link on a login page gives a "404 Not Found" error.*

Stage	What happens	Example
1. Identified	During test execution, the tester sees that the actual result is different from the expected result. The tester repeats the test to confirm it and checks that it is not already reported.	Clicking "Forgot Password" shows a 404 page instead of the reset form.
2. Recorded	A defect report is logged in the tool with summary, steps, expected/actual result, environment, severity, priority and screenshots. Status = **New**.	DEF-210 logged; severity Major, priority High.
3. Assigned	In triage, the lead validates the defect and assigns it to a developer with a target date. Status = **Assigned**. Invalid or duplicate defects are rejected here.	Assigned to the login-module developer, due in 2 days.
4. Fixed	The developer reproduces the defect, finds the root cause, corrects the code, unit-tests it and checks it in. Status moves to **Open** and then **Fixed**.	Wrong URL mapping corrected; fix included in build 2.4.
5. Retested	The tester verifies the fix on the new build by re-running the failed test and related regression tests. Status = **Retest**.	Link now opens the reset page; login and registration still work.
6. Closed / Reopened	If the fix works, the status is changed to **Verified** and then **Closed** with comments. If not, it is **Reopened** and returned to the developer.	DEF-210 closed after successful verification.

After closure, the defect data are used for metrics, root cause analysis and process improvement, so that the cycle of defect management is complete.

Q12. What are Test Metrics? Explain their importance in software testing.

Test metrics are quantitative measures used to monitor, evaluate and control testing activities and the quality of the product being tested. They convert raw testing data (number of test cases, defects, hours spent) into meaningful figures such as percentages and ratios. For example: *"90% of test cases executed, of which 95% passed"*, or *"defect density is 2 defects per KLOC"*.

Importance of test metrics

- **Progress tracking:** Shows how much of the planned testing is complete.
- **Quality assessment:** Indicates the stability and quality of the software.
- **Effectiveness measurement:** Shows how good the testing is at finding defects.
- **Efficiency measurement:** Shows how well time, effort and resources are used.
- **Decision support:** Helps management decide on release, extra testing or resource changes.
- **Early problem detection:** Highlights risky modules, delays and bottlenecks in time.
- **Estimation and planning:** Historical metrics improve future estimation of effort and schedule.
- **Process improvement:** Trends reveal weak areas in the testing and development process.

- **Communication:** Gives stakeholders objective, factual information instead of opinions.
- **Accountability:** Sets measurable targets for the team.

Q13. Explain different types of Test Metrics used to measure testing activities.

A. Classification by purpose

Type	What it measures	Examples
Process metrics	Efficiency and effectiveness of the testing process.	Test execution rate, defect detection percentage, defect removal efficiency, test automation percentage.
Product metrics	Quality and characteristics of the software.	Defect density, defects by severity, defects by module, mean time to failure.
Project metrics	Progress, cost and schedule of the testing project.	Planned vs. actual effort, schedule variance, cost variance, test cost per case.

B. Classification by nature of data

- **Absolute (base) metrics:** raw counts collected directly, such as total test cases, test cases executed/passed/failed/blocked, total defects found, defects by severity and test hours spent.
- **Derived (calculated) metrics:** values computed from base metrics, such as percentages, ratios and averages.

C. Commonly used test metrics

Metric	Formula	Measures
Test Case Execution %	(Test cases executed ÷ Total test cases) × 100	Progress
Test Case Pass %	(Passed ÷ Executed) × 100	Product quality
Test Case Fail %	(Failed ÷ Executed) × 100	Product quality
Test Case Blocked %	(Blocked ÷ Total test cases) × 100	Obstacles
Requirements Coverage	(Requirements with tests ÷ Total requirements) × 100	Completeness
Test Coverage (code)	(Code covered by tests ÷ Total code) × 100	Thoroughness
Defect Density	Total defects ÷ Size (KLOC / function points)	Code quality
Defect Detection Percentage	(Defects found by testing ÷ Total defects found) × 100	Test effectiveness
Defect Leakage	(Defects found after release ÷ Defects found before release) × 100	Missed defects
Test Execution Rate	Test cases executed ÷ Time (days/hours)	Speed
Automation Coverage	(Automated test cases ÷ Total test cases) × 100	Automation level
Effort / Schedule Variance	((Actual − Planned) ÷ Planned) × 100	Project control

Q14. What is Test Reporting? Explain the importance of test reports in software testing.

Test reporting is the activity of collecting, analysing and communicating information about testing activities and results to stakeholders. A **test report** is the document (or dashboard) that summarises what was tested,

how the testing went, what defects were found, and an assessment of the quality and readiness of the software for release. Test reports are produced during testing (progress/status reports) and at the end (test summary report).

Importance of test reports

- **Provide visibility:** Stakeholders get a clear picture of testing progress and product quality.
- **Support release decisions:** Management can decide whether the software meets the exit criteria and is ready to go live.
- **Communicate defects and risks:** Open defects, risks and untested areas are highlighted for action.
- **Track progress against the plan:** Show planned vs. actual execution, delays and bottlenecks.
- **Demonstrate coverage:** Prove which requirements and features have been verified.
- **Evidence and accountability:** Serve as proof of testing for audits, customers and regulators.
- **Basis for improvement:** Lessons learned and metrics help to improve future projects.
- **Historical reference:** Help in estimation, planning and regression testing in later releases.
- **Build confidence:** Give customers and management confidence in the product quality.

Q15. What are the main contents of a Test Report? Explain.

The test summary report is usually prepared as per IEEE 829. Its main contents are:

Content	Explanation
Report identifier and title	Project name, report ID, version, date and author.
Introduction / Purpose	Objective of the report and a short description of the project and the application tested.
Scope of testing	Features and modules tested, and those not tested (with reasons).
Test approach and environment	Test levels and types performed, tools used, hardware/software, browsers and test data used.
Test summary / results	Number of test cases planned, executed, passed, failed, blocked and not run.
Defect summary	Total defects found, by severity, priority, status (open/fixed/closed) and module; details of critical open defects.
Test metrics	Execution %, pass %, defect density, defect leakage, coverage, etc.
Deviations from the plan	Differences between planned and actual scope, schedule or effort, with reasons.
Risks, issues and limitations	Remaining risks, known issues, constraints and workarounds.
Exit criteria evaluation	Whether the exit criteria in the test plan were met.
Conclusion and recommendation	An overall quality assessment and a recommendation: release / release with conditions / do not release.
Lessons learned	What went well, what did not, and suggestions for improvement.
Appendices and approvals	Detailed results, graphs, references, and sign-off by test lead, project manager and stakeholders.

Q16. Explain different Test Reporting Techniques used in software testing.

Test reporting techniques are the different ways of presenting and communicating test information. The technique chosen depends on the audience and the purpose.

Technique	Description
Test status / progress reports (daily, weekly)	Short periodic reports showing test cases executed, passed/failed, new and open defects, blockers and plans for the next period. Mostly used by the test team and project manager.
Test summary report	A formal final report at the end of a test cycle or project giving the overall results, metrics, evaluation of exit criteria and the release recommendation.
Defect reports	Reports specifically on defects: defect status, severity and priority distribution, ageing and open critical defects.
Dashboards	Real-time web-based views (e.g., Jira, TestRail dashboards) with live charts and counters of test and defect status.
Graphical reporting (charts and graphs)	Bar charts, pie charts, line charts, burn-down charts and defect-trend graphs that show data visually for easy understanding.
Traffic-light (RAG) status reporting	Colour-coded status: Red = serious problems, Amber = at risk, Green = on track. Gives a quick overview for managers.
Metrics-based reporting	Reports built around key metrics such as execution %, defect density, DRE and leakage, often with comparisons against targets.
Traceability matrix reports	Show requirement-to-test-case-to-defect mapping, proving requirement coverage.
Verbal / meeting-based reporting	Daily stand-ups, status meetings and defect-triage meetings, giving instant communication and discussion.
Email / tool-generated reports	Automated or manual emails and exported reports (PDF/Excel) from test management and CI/CD tools.

A good report is **accurate, timely, concise, objective, and suitable for its audience** (details for testers, summaries and risks for management).

Q17. Differentiate between Test Metrics and Test Reports.

Basis	Test Metrics	Test Reports
Meaning	Quantitative measures of testing activities and product quality.	Documents that communicate testing status, results and conclusions.
Nature	Numbers, ratios and percentages.	Narrative plus data, tables and charts.
Purpose	To measure, analyse and compare progress, quality, effectiveness and efficiency.	To inform stakeholders and support decisions such as release.
Content	Individual measurements, e.g., pass % or defect density.	Scope, results, defects, metrics, risks, conclusion and recommendations.
Relationship	Raw input and building blocks for reports.	Metrics are presented and interpreted in the report.
Audience	Mainly test team and management for analysis.	Wider audience: management, developers, clients and stakeholders.

Basis	Test Metrics	Test Reports
Timing	Collected continuously during testing.	Prepared periodically and at the end of a test cycle.
Interpretation	Needs interpretation to be meaningful.	Already includes analysis and recommendations.
Example	Test case pass rate = 92%.	Test summary report stating the build is ready for release with 2 minor open defects.

Q18. Explain how defect metrics can be used to measure the quality of software.

Defects are the clearest evidence of poor quality, so **defect metrics** give an objective way to judge the quality of a product, the effectiveness of testing and the readiness for release. The main defect metrics are:

Defect metric	How it is calculated	What it tells about quality
Defect Density	Total defects ÷ Size (KLOC or function points)	High density means poor code quality or a risky module.
Defect Distribution by Severity	Number of defects at each severity level	Many critical/major defects indicate unstable software.
Defect Distribution by Module	Defects per module/component	Reveals defect-prone modules needing more testing or redesign (defect clustering).
Defect Trend (found vs. fixed)	Defects found and closed per day/week	A falling trend of new defects suggests that the product is stabilising.
Defect Removal Efficiency (DRE)	Defects removed before release ÷ (before + after release) × 100	High DRE means high-quality development and testing.
Defect Leakage	Defects found after release ÷ Defects found in testing × 100	High leakage means testing missed many defects.
Defect Age	Closed date − Reported date	Long ageing indicates slow resolution or process problems.
Reopen Rate	Reopened defects ÷ Fixed defects × 100	A high rate indicates poor fixes or weak unit testing.
Rejection / Invalid Rate	Rejected defects ÷ Total reported × 100	Indicates unclear requirements or weak defect reporting.
Open Defect Count	Unresolved defects (especially critical)	Used in release readiness and exit criteria.

Worked example

Suppose an application of **20 KLOC** has **80 defects** found during testing and **10 defects** found by users after release.

- **Defect density** = 80 ÷ 20 = **4 defects per KLOC**.
- **DRE** = 80 ÷ (80 + 10) × 100 = **88.9%** (good, but there is room to improve).
- **Defect leakage** = 10 ÷ 80 × 100 = **12.5%**.

If a few modules contain most of the defects, extra testing and code review should be applied there. If new defect discovery has levelled off and no critical defects remain open, the quality is stable enough for release. In this way, defect metrics objectively show the current quality, predict risk and guide improvement.

Q19. Explain how Test Metrics can be used to measure the effectiveness and efficiency of a testing team.

Effectiveness means *how well* the testing team achieves its goal of finding defects and covering the software (doing the right things). **Efficiency** means *how economically* it does this in terms of time, effort and cost (doing things right).

Measuring effectiveness

Metric	Calculation	Indicates
Defect Detection Percentage (DDP)	(Defects found by the team ÷ Total defects found, including after release) × 100	Share of defects caught by the team
Defect Leakage	(Defects found after release ÷ Defects found in testing) × 100	Defects missed (lower is better)
Requirement / Test Coverage	(Requirements or code covered ÷ Total) × 100	Completeness of testing
Test Case Effectiveness	(Defects found ÷ Test cases executed) × 100	Defect-finding power of test cases
Defect Rejection Ratio	Rejected defects ÷ Total reported defects × 100	Quality of defect reporting (lower is better)
Severity of defects found	Share of critical/major defects found by the team	Ability to detect important defects

Measuring efficiency

Metric	Calculation	Indicates
Test Execution Rate	Test cases executed ÷ Hours (or days)	Speed of testing
Test Design Productivity	Test cases written ÷ Person-days	Productivity of test design
Defects found per hour / per tester	Defects ÷ Testing effort	Productivity
Cost per defect / per test case	Testing cost ÷ Defects (or test cases)	Cost efficiency
Schedule and Effort Variance	((Actual − Planned) ÷ Planned) × 100	Accuracy of estimation and control
Automation Coverage	(Automated test cases ÷ Total) × 100	Reduction of manual effort
Defect Turnaround / Retest time	Time taken to retest and close fixed defects	Responsiveness of the team

Q20. How can metrics be used to identify problems and improve testing effectiveness? Explain.

Metrics work like a health check of the testing process. When they are collected regularly and compared with expected values, abnormal readings point out problems and the data helps to decide what to improve.

Identifying problems from metrics

Metric observation	Possible problem	Improvement action
High defect leakage / low DRE	Test cases miss defects; poor coverage.	Add test cases for missed areas, apply better design techniques (EP, BVA), strengthen regression and reviews.
Low test coverage	Some requirements or code are untested.	Use a traceability matrix and add tests for uncovered items.
Defects clustered in a few modules	Complex or poorly written modules (defect clustering).	Increase testing and code reviews in those modules; consider redesign.
High reopen rate	Poor fixes or weak unit testing; unclear defect reports.	Improve defect reports, developer unit testing and retest procedures.
High rejection rate	Requirements are unclear or testers misunderstand them.	Clarify requirements; train testers; improve communication.
High defect age	Slow resolution; delays in triage or assignment.	Improve triage, set service-level targets and escalate delays.
Low execution rate / many blocked cases	Environment problems, build instability or low productivity.	Fix the environment, improve build quality, add resources or automation.
Large schedule or effort variance	Estimates are unrealistic; scope creep.	Improve estimation using past data and control scope changes.
Defects found late in the cycle	Late test design or lack of early reviews.	Shift testing left: review requirements and design early; start test design sooner.
Low automation coverage	Too much manual repetitive effort.	Automate regression and repetitive tests.

Steps to improve effectiveness using metrics

1. **Set goals:** Decide what should be improved (for example, reduce defect leakage to below 5%).
2. **Choose relevant metrics:** Select a few metrics linked to the goals (Goal–Question–Metric approach).
3. **Collect data regularly:** Use tools to record data accurately and consistently.
4. **Analyse and compare:** Compare with targets, earlier releases and benchmarks; study trends over time.
5. **Find root causes:** Investigate why the metric is poor (root cause analysis, retrospectives).
6. **Take corrective action:** Update test cases, tools, training and processes.
7. **Re-measure and review:** Check whether the metrics improve, and repeat the cycle for continuous improvement.